

## Midterm Exam

Print your name and Knox login *neatly* in the spaces provided below. Please print legibly.

|             |
|-------------|
| Name:       |
| Knox login: |

This is an *open note* exam. You are permitted to use written notes, but not a computer. Students whose native language is not English may use a dictionary. *Nothing else is permitted.*

Do all the problems in this booklet. If you get stuck, it may be beneficial to skip that problem and come back to it.

If a problem is unclear to you, include a brief explanation of how you interpret it.

*Show your work!* You will not get partial credit if we cannot figure out how you got your answer.

| Question | Points | Score |
|----------|--------|-------|
| 1        | 20     |       |
| 2        | 20     |       |
| 3        | 20     |       |
| 4        | 20     |       |
| 5        | 20     |       |
| Total    | 100    |       |

**1. Running times (20 points).**

Give the running time in Big-O notation for each of the following. Assume that all arrays, lists, etc have size  $n$ . If you want partial credit, include a brief explanation for your answers. Assume that `ArrayList` is implemented the way we saw in class and that `LinkedList` is a doubly-linked list with a tail pointer.

- (a) The `clear` method of `ArrayList<String>`. Recall that this method empties the list.
- (b) The `equals` method of `LinkedList<String>`, which determines whether two lists contain the same elements (in the same order).
- (c) The `remove` method of an iterator on a `LinkedList<String>`. Recall that this method removes the most-recently-returned value from the list.

(d) //A and B are arrays of integers

```
int max = 0;
for(int i=0; i < A.length; i++) {
    int num = 0;
    for(int j=0; j < B.length; j++) {
        if(A[i] == B[j])
            num++;
    }
    if(num > max)
        max = num;
}
```

(e) //A is an array of Strings

```
ArrayList<String> list = new ArrayList<String>();
for(int i=0; i < A.length; i++) {
    if(!list.contains(A[i]))
        list.add(A[i]);
}
```



**3. Testing (20 points).**

Consider a possible List method `replace`, which takes two values `v1` and `v2`. The method changes the first occurrence of `v1` into `v2`. If `v1` does not appear in the list, the method throws a `NoSuchElementException`.

Write JUnit test cases for this method. Test against `OurList`, an interface implemented by both `OurArrayList` and `OurLinkedList`. You can assume that `OurList` has the following methods: `add` (adds to end), `size`, `get`, and `contains`.

```
//import statements not shown
```

```
public class TestList {
    private OurList<String> list;

    @Before
    public void setUp() {
        list = new OurArrayList<String>();
    }

    @Test
    public void testStuff() {
```

**4. ArrayList methods (20 points).**

Write an implementation of `replace` for `OurArrayList`, our array-based list implementation. This is the same method described on the previous page. It takes two values `v1` and `v2`, changing the first occurrence of `v1` into `v2`. If `v1` does not appear in the list, the method throws a `NoSuchElementException`.

**5. LinkedList methods (20 points).**

Write an implementation of `removeRange` for `OurLinkedList`, our linked memory list implementation. This is the same method described in the two previous problems. It takes two values `v1` and `v2`, changing the first occurrence of `v1` into `v2`. If `v1` does not appear in the list, the method throws a `NoSuchElementException`.